

A Knowledge-Grounded Multi-Agent Reasoning System for Electrical Engineering Applications and Policy Navigation

A Project Report

Submitted in Partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

ELECTRICAL AND ELECTRONICS ENGINEERING

By

Mehul Jain (Roll No. 522206)

Krishna Tayal (Roll No. 522152)

Ramasish Parida (Roll No. 522232)

Supervisor:

Sri Phani Krishna Karri



**DEPARTMENT OF ELECTRICAL ENGINEERING
NATIONAL INSTITUTE OF TECHNOLOGY ANDHRA PRADESH**

April 2026

Approval Sheet

This project work entitled “A Knowledge-Grounded Multi-Agent Reasoning System for Electrical Engineering Applications and Policy Navigation” by Mehul Jain (Roll No. 522206), Krishna Tayal (Roll No. 522152), and Ramashish Parida (Roll No. 522232) is approved for the award of the degree of Bachelor of Technology in Electrical and Electronics Engineering from the National Institute of Technology Andhra Pradesh.

Supervisor

Dr. Sri Phani Krishna Karri
Head of the Department
Department of Electrical Engineering
National Institute of Technology Andhra Pradesh

Head of the Department

Dr. Sri Phani Krishna Karri
Head of the Department
Department of Electrical Engineering
National Institute of Technology Andhra Pradesh

Date: _____

Place: _____

Declaration

We hereby declare that this written submission represents our ideas in our own words. Wherever others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented, fabricated, or falsified any idea, data, fact, or source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and may also invite penal action from the concerned sources.

Mehul Jain

Roll No.: 522206

Signature: _____

Date: _____

Krishna Tayal

Roll No.: 522152

Signature: _____

Date: _____

Ramasish Parida

Roll No.: 522232

Signature: _____

Date: _____

Certificate

This is to certify that the project report entitled “A Knowledge-Grounded Multi-Agent Reasoning System for Electrical Engineering Applications and Policy Navigation” is a bona fide record of the work carried out by Mehul Jain (Roll No. 522206), Krishna Tayal (Roll No. 522152), and Ramashish Parida (Roll No. 522232) submitted to the Department of Electrical Engineering, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Electrical and Electronics Engineering at the National Institute of Technology Andhra Pradesh during the academic year 2025–2026.

The above-mentioned students have worked under my guidance and supervision and have fulfilled the requirements for the submission of this project report, which, to the best of my knowledge, has attained the requisite standard. The results embodied in this report have not been submitted to any other University or Institute for the award of any degree or diploma.

Supervisor

Dr. Sri Phani Krishna Karri
Head of the Department
Department of Electrical Engineering
National Institute of Technology Andhra Pradesh

Head of the Department

Dr. Sri Phani Krishna Karri
Head of the Department
Department of Electrical Engineering
National Institute of Technology Andhra Pradesh

Acknowledgements

We would like to express our sincere gratitude to Dr. Sri Phani Krishna Karri, Project Guide and Head of the Department of Electrical and Electronics Engineering, for his invaluable guidance, constant encouragement, and support throughout the course of this project. His insights and feedback helped us refine the technical direction of the work and present it in a more structured and meaningful manner.

We are grateful to the faculty members of the Department of Electrical and Electronics Engineering, National Institute of Technology Andhra Pradesh, for their support and for providing us with the necessary resources and facilities to complete this project. Their academic environment and encouragement made it possible for us to explore the interdisciplinary aspects of electrical engineering, natural language processing, knowledge graphs, and tool-based reasoning.

We also extend our heartfelt thanks to our peers and colleagues who contributed their ideas, suggestions, and feedback during the development of this project. Their discussions and support helped us improve both the technical depth and the presentation quality of the work.

Mehul Jain

522206

Krishna Tayal

522152

Ramasish Parida

522232

Abstract

The deployment of Large Language Models (LLMs) as conversational agents for technical assistance and policy navigation is often hindered by their propensity for hallucination and lack of numerical precision. While LLMs excel in natural language understanding, their reliability diminishes significantly in domains requiring exact mathematical computations, such as electrical engineering, and in policy navigation. This project addresses these critical limitations by developing a knowledge-grounded, multi-agent reasoning framework that integrates LLM-based orchestration with specialized deterministic tools, specifically a MATLAB execution engine and a structured Knowledge Graph.

The proposed hybrid architecture delegates distinct tasks to specialized agents to ensure both factual accuracy and computational integrity. A central intelligent orchestrator dynamically classifies user queries and routes them accordingly. For engineering tasks involving complex symbolic manipulation, numerical analysis, or plotting, the system utilizes a MATLAB execution agent that autonomously generates, verifies, and executes code, thereby bypassing the unreliable token-prediction mechanism of standard LLMs. For queries involving policy navigation, the system grounds its responses in a Neo4j-based Knowledge Graph, which maps hierarchical policy dependencies and facilitates multi-hop reasoning, additionally supplemented by iterative web searches for external context.

Evaluated against conventional generative models, the dual-agent framework demonstrates a marked improvement in performance. The MATLAB agent achieves consistent, high-fidelity results for multi-step mathematical problems, while the Knowledge Graph ensures precise, traceable retrieval of policy facts without semantic fragmentation. By effectively separating linguistic reasoning from deterministic execution and structured retrieval, this project presents a robust and scalable solution to the trust gap in AI, bridging the divide between fluid conversational interfaces and the rigorous demands of engineering and policy navigation.

Key Words: Multi-Agent Systems; Large Language Models; Knowledge Graph; MATLAB Integration; Web Search; Electrical Engineering; Policy Navigation; Retrieval-Augmented Generation; Tool-Augmented Reasoning.

TABLE OF CONTENTS

Approval Sheet	i
Declaration	ii
Certificate	iii
Acknowledgements	iv
Abstract	v
List of Symbols	xi
List of Acronyms	xii
I Introduction	1
1.1 General	1
1.2 Problem being addressed	2
1.3 Aim	2
1.4 Objectives	2
1.5 State of the Art	3
1.6 Scope of the work	4
1.7 Outline of the Chapters	4
II LITERATURE REVIEW	6
2.1 General	6
2.2 Review of Existing Methods	6
2.2.1 Retrieval-Augmented Generation (RAG)	6

2.2.2	Knowledge Graph-Based Question Answering (KGQA)	7
2.2.3	Tool-Augmented LLMs	7
2.2.4	Multi-Agent Systems (MAS)	7
2.2.5	MATLAB and Symbolic Computation in AI Systems	7
2.3	Summary	8
III	Proposed System Architecture	9
3.1	The Unified Engineering Team: A System Overview	9
3.2	The Orchestrator: Task Triage and Management	9
3.3	The Knowledge Agent: Accessing Policy Navigation Memory	10
3.4	The MATLAB Agent: Expert Mathematical Solving	10
IV	Methodology	12
4.1	Knowledge Graph Design and Construction	12
4.1.1	Extraction Strategy	12
4.1.2	Graph Population	12
4.1.3	Grounding and Entity Resolution	13
4.2	Query Processing Pipeline	15
4.3	Multi-Agent Coordination	15
4.4	Validation and Feedback Loops	15
V	Implementation	17
5.1	Technical Stack	17
5.2	Tool Integration and Workflow	17
VI	Experimental Setup and Results	19
6.1	Testing with MATLAB as an Execution Tool	19
6.1.1	Evaluation Objectives	19
6.1.2	Dataset and Query Design	19
6.1.3	Performance Benchmarking	20
6.1.4	Computational Efficiency	21
6.2	Testing Knowledge Graph Retrieval with NIT Andhra Pradesh Policies	21

6.2.1	Evaluation Objectives	21
6.2.2	Dataset and Query Design	21
6.2.3	Retrieval Performance and Analysis	22
6.2.4	Error Analysis and Key Findings	23
VIIDiscussions		25
7.1	Conclusions	25
7.2	Future Scope	26
BIO-DATA		29

LIST OF FIGURES

III.1 Detailed System Architecture and Workflow	11
IV.1 Knowledge Graph Flowchart	14
VI.1 Comparative performance across difficulty levels	20
VI.2 Execution time comparison and inefficiency gap vs complexity	21
VI.3 Knowledge Graph retrieval benchmark across specific query types	22
VI.4 MATLAB plots of various questions	24

LIST OF TABLES

I.1	Comparison of Retrieval Approaches in Technical Domains	4
II.1	Summary of Recent Literature on AI Reasoning Systems	8
III.1	System Components: Analogies and Responsibilities	10
V.1	Technical Stack Components	17
VI.1	Distribution of mathematical questions by topic and difficulty	20
VI.2	Representative Knowledge Graph queries mapping policy navigation	22

List of Symbols

Y_{bus}	Bus admittance matrix
Z_{bus}	Bus impedance matrix
V	Voltage vector
I	Current injection vector
P	Real power
Q	Reactive power
S	Complex power
K	Knowledge graph node or edge count in a substructure

List of Acronyms

AI	Artificial Intelligence
API	Application Programming Interface
CSV	Comma-Separated Values
Cypher	Query language for Neo4j graphs
DPR	Dense Passage Retrieval
EE	Electrical Engineering
EEE	Electrical and Electronics Engineering
ERP	Enterprise Resource Planning
GPT	Generative Pre-trained Transformer
IPC	Inter-Process Communication
JSON	JavaScript Object Notation
KG	Knowledge Graph
KGQA	Knowledge Graph Question Answering
LLM	Large Language Model
MAS	Multi-Agent Systems
MATLAB	Matrix Laboratory
Neo4j	Graph database platform
NIT	National Institute of Technology
NLP	Natural Language Processing
OCR	Optical Character Recognition
RAG	Retrieval-Augmented Generation
SCADA	Supervisory Control and Data Acquisition
SQL	Structured Query Language
UML	Unified Modeling Language

I Introduction

1.1 General

The evolution of large language models (LLMs) has marked a paradigm shift in artificial intelligence, moving beyond simple token prediction toward complex reasoning and problem-solving capabilities [1, 2]. To understand the significance of this development, one must first consider how these models function at a fundamental level. Unlike traditional software that follows a fixed set of deterministic rules, LLMs are statistical engines trained on vast datasets to predict the most likely “next token” in a sequence [2]. This probabilistic methodology allows them to generate remarkably human-like text and perform creative synthesis across a wide array of topics. However, it also introduces a critical vulnerability known as “hallucination,” where the model produces outputs that are semantically plausible but factually or numerically incorrect [3, 4].

In high-stakes professional domains such as electrical engineering and policy navigation, these hallucinations are more than mere inconveniences; they represent a fundamental barrier to the adoption of artificial intelligence [5]. In circuit design or signal processing, for instance, a minor sign error or a miscalculated iterative value can lead to catastrophic system failure [6]. Similarly, in policy navigation, misinterpreting a single clause regarding promotion criteria or disciplinary actions can have severe legal and administrative consequences. This project addresses these risks by presenting a hybrid multi-agent system designed to bridge the gap between the fluid reasoning of LLMs and the rigid requirements of technical accuracy.

By re-positioning the language model as an **intelligent orchestrator** [7] rather than a primary knowledge source, the system leverages deterministic external engines for specialized tasks [8]. Two core technologies are used to ground the model’s responses in reality: MATLAB for complex symbolic computation and Knowledge Graphs (KGs) for structured retrieval [9]. This architecture ensures that technical analysis remains anchored in verified mathematical logic and formal policy structures. Instead of relying on its internal statistical memory, the model identifies the technical requirements of a user’s query and delegates the work to a specialized agent that can provide a verifiable answer.

1.2 Problem being addressed

Current large language models face two primary “bottlenecks” that limit their usefulness in specialized professional work: the Trust Gap in numerical computation and the Contextual Fragmentation in document retrieval. The Trust Gap refers to the inherent inability of an LLM to guarantee accuracy in multi-step calculations or symbolic manipulation [10]. Because the model is predicting the most likely next word rather than executing logical operations, it frequently shortcuts complex equations or produces values by association rather than calculation [11]. In the domain of electrical engineering, where problems such as Newton-Raphson iterations or Adjoint Matrix calculations require absolute precision, this predictive nature is fundamentally unreliable [11]. Even state-of-the-art models often deviate in long computational chains, rendering their engineering advice invalid for professional use.

The second bottleneck, Contextual Fragmentation, arises from the limitations of conventional Retrieval-Augmented Generation (RAG) systems [9]. RAG is often described as giving the AI an “open-book exam” where it can look up relevant text before generating an answer. However, traditional RAG relies on semantic similarity, which treats documents as a collection of independent text chunks [12]. This approach fails when navigating policy documents that are highly structured and hierarchical [12]. Documents like university handbooks or regulatory frameworks contain cross-references, exceptions, and conditional logic spread across multiple chapters. A simple keyword search might retrieve a specific rule but miss the overarching exception found in another section. This lack of structural understanding leads to incomplete or misleading interpretations of an organization’s rules.

Furthermore, there is a distinct lack of provenance in current LLM applications [9]. When a model provides an answer, it is often impossible for a user to audit exactly where that information came from or how a specific numerical result was derived. This lack of traceability makes the system a “black box,” which is unacceptable in policy navigation and engineering environments where accountability is paramount [5]. This project directly addresses these weaknesses by developing an architecture that strictly prevents the language model from guessing at quantitative or structurally complex answers, instead routing such tasks to deterministic external systems that provide clear logical traces.

1.3 Aim

The primary aim of this research is to develop a robust, knowledge-grounded multi-agent reasoning system that bridges the gap between fluid natural language interaction and the rigorous requirements of engineering computation and policy navigation.

1.4 Objectives

The main objectives of this research are:

- To design and implement a multi-agent framework that enforces a **Code-as-Reasoning** paradigm for tech-

nical domains and a **Graph-as-Context** approach for policy navigation queries.

- To develop an intent-detection mechanism that automatically routes queries between mathematical computation, structured relationship-based retrieval, and general conversational synthesis.
- To integrate a MATLAB execution environment for autonomous code generation, execution, and verification, ensuring deterministic accuracy in engineering tasks.
- To transform unstructured regulatory documents into a directed knowledge graph, enabling multi-hop reasoning and consistent retrieval of complex policy dependencies.
- To implement iterative self-correction and feedback loops that validate computational results and refine retrieval accuracy, providing a transparent and verifiable audit trail.

1.5 State of the Art

Existing approaches to improving language model reliability broadly fall into four categories: Retrieval-Augmented Generation (RAG), Knowledge Graph-based Question Answering (KGQA), Tool-Augmented LLMs, and Multi-Agent Frameworks. Retrieval-Augmented Generation has become the industry standard for grounding models in external text data. By retrieving relevant snippets from a vector database before generation, RAG helps reduce hallucinations for factual queries. However, as previously discussed, RAG remains limited when dealing with hierarchical policy documents or problems requiring precise, multi-step calculations because it treats information as disjointed text fragments rather than a structured logical network.

Knowledge Graph-based Question Answering (KGQA) attempts to solve the structural limitation of RAG by representing knowledge as explicit triplets (subject-predicate-object) within a graph [13]. This allows for deep multi-hop reasoning, where a system can traverse multiple related nodes to find an answer. While KGQA is highly effective for policy navigation and relationship mapping, it is rarely combined with dynamic execution engines for technical domains, leaving a gap in systems that require both logical relationship mapping and numerical calculation.

Tool-Augmented LLMs, such as the Toolformer model [14], have introduced the idea of teaching a model to use external APIs for specialized tasks. These models can call calculators, search engines, or translation tools to augment their internal knowledge. While promising, most implementations are designed for simple tool calls rather than the deep, iterative integration required for engineering software like MATLAB. Multi-agent systems, such as the AutoGen framework [8], further extend this by using multiple specialized agents to decompose complex tasks. However, few such frameworks have been tailored specifically for the simultaneous dual demands of high-precision symbolic computation in electrical engineering and relational logic in policy navigation. This project advances the state of the art by unifying these sparse methodologies into a single, domain-focused hybrid architecture.

Table I.1: Comparison of Retrieval Approaches in Technical Domains

Approach	Handles Numerical Computation	Multi-hop / Hierarchical Reasoning	Traceability and Provenance	Suitability for Engineering and Policy
Standard RAG	Low	Low	Medium	Limited
Knowledge Graph QA	Low	High	High	Good for Policy
Tool-Augmented LLMs	High	Medium	High	Strong for Engineering
Multi-Agent Systems	Medium	High	Medium	Promising
Hybrid Orchestration (Proposed)	High	High	High	Excellent

1.6 Scope of the work

This project delivers a complete hybrid multi-agent framework that bridges large language models with symbolic computation and structured knowledge systems. The scope of this research is specifically focused on two critical professional domains that demand high levels of accuracy and traceability: electrical engineering and policy navigation. In the electrical engineering domain, the research focuses on developing a MATLAB-based agent capable of high-fidelity analysis for circuit design, signal processing, and iterative numerical methods. This includes creating a robust pipeline for automated code generation, linting, execution, and iterative self-correction based on runtime feedback.

In the policy navigation domain, the research addresses the challenge of managing complex regulatory documents, such as university statutes and student handbooks. The scope includes the development of methodologies for digitising these unstructured documents into directed knowledge graphs, enabling the system to perform accurate traversals for rules, procedures, and exceptions. By focusing on these two distinct but equally demanding areas, the project demonstrates the versatility of the hybrid architecture across both symbolic math and relational logic.

The research also introduces a practical methodology for legacy document conversion and a “code-as-reasoning” pipeline for translating abstract problems into executable scripts. Evaluation is performed against a custom benchmark of complex mathematical problems and multi-layer policy queries, measuring both the consistency of the logical trace and the accuracy of the final computational result. Theoretical contributions are centered on the design of the orchestration layer and the evaluation of the closed-loop self-correction mechanism. The resulting framework is designed to be model-agnostic, providing a scalable and trustworthy foundation for AI deployment in professional and academic environments.

1.7 Outline of the Chapters

The contents of this report have been organized into the following chapters:

I: Introduction — This chapter introduces the background of the project, defines the problem addressed, states the aim and objectives, reviews the state of the art, and outlines the scope and organization of the report.

II: Literature Review — This chapter presents a comprehensive and critical review of published literature related

to Large Language Models, Knowledge Graphs, multi-agent systems, and tool-augmented reasoning within engineering domains, identifying research gaps that motivate this work.

III: Proposed System Architecture — This chapter presents the conceptual design of the unified multi-agent framework, detailing the functions of the Orchestrator, the Knowledge Agent, and the MATLAB Agent.

IV: Methodology — This chapter explains the methods used for system development, including multimodal extraction, Knowledge Graph construction, and the iterative validation loops for computational tasks.

V: Implementation — This chapter outlines the technical stack, encompassing the Python-based framework, Neo4j graph database, Ollama inference engine, and the inter-process communication bridge for MATLAB.

VI: Experimental Setup and Results — This chapter presents the comparative benchmarking of the system against traditional language models for both mathematical calculations and policy navigation retrieval.

VII: Conclusions and Future Scope — This chapter summarizes the main conclusions drawn from the project work and suggests directions for future research.

II LITERATURE REVIEW

2.1 General

A comprehensive review of existing literature was conducted to understand the current advancements in Large Language Models (LLMs), Knowledge Graphs (KGs), multi-agent systems, and tool-augmented reasoning within engineering domains. The literature survey was performed using major academic databases such as IEEE Xplore, ScienceDirect, SpringerLink, and Google Scholar. Keywords including “knowledge-grounded systems”, “multi-agent LLMs”, “retrieval augmented generation”, “engineering AI systems”, and “MATLAB-based computation” were used. Recent works from 2018 to 2025 were prioritized to capture the latest developments in the field.

The purpose of this literature review is to identify limitations in existing approaches, particularly in handling domain-specific reasoning, mathematical precision, and policy navigation, which motivates the development of the proposed system.

2.2 Review of Existing Methods

2.2.1 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) has emerged as a prominent approach to enhance the factual accuracy of LLMs by incorporating external knowledge sources. Lewis et al. [9] introduced the RAG framework, which combines dense vector retrieval with generative models to improve knowledge grounding. Their approach demonstrated significant improvements in open-domain question answering tasks.

However, several limitations exist in vector-based retrieval methods. Karpukhin et al. [15] proposed dense passage retrieval (DPR), which improves retrieval performance but still struggles with highly structured and cross-referenced technical documents. These methods lack the ability to capture explicit relationships between entities, making them less suitable for policy-based reasoning and engineering applications.

Recent works such as [16] have attempted to improve retrieval quality using hybrid approaches, but challenges remain in multi-hop reasoning and precise domain-specific retrieval.

2.2.2 Knowledge Graph-Based Question Answering (KGQA)

Knowledge Graphs provide structured representations of entities and relationships, making them highly suitable for domain-specific reasoning. Wang et al. [17] explored KG-based question answering systems, demonstrating improved interpretability and accuracy for structured queries.

Further advancements by Sun et al. [18] introduced embedding-based approaches for knowledge graph reasoning, enabling efficient link prediction and entity relationship inference. These approaches improve scalability but often lack integration with natural language understanding.

In enterprise and policy-based systems, knowledge graphs have shown strong performance in handling multi-hop queries and rule-based reasoning. However, their limitation lies in handling unstructured data and integrating dynamic external knowledge sources.

2.2.3 Tool-Augmented LLMs

Tool-augmented LLMs enhance reasoning capabilities by enabling models to interact with external computational tools. Schick et al. [14] proposed Toolformer, which allows LLMs to autonomously decide when to use external APIs for improved reasoning.

Similarly, OpenAI's function-calling frameworks enable structured tool invocation, improving reliability in mathematical and logical tasks. These systems are particularly effective in engineering applications where numerical precision is critical.

However, standalone tool usage lacks coordination when solving complex multi-step problems, highlighting the need for structured orchestration mechanisms.

2.2.4 Multi-Agent Systems (MAS)

Multi-agent systems have gained traction as a solution for decomposing complex tasks into manageable subtasks. Park et al. [19] demonstrated generative agents capable of collaborative reasoning and planning.

In LLM-based systems, multi-agent frameworks enable modular design, where each agent specializes in a specific task such as retrieval, computation, or synthesis. This improves scalability and interpretability.

Despite these advantages, challenges remain in agent coordination, task routing, and conflict resolution, especially in hybrid systems involving both symbolic and neural components.

2.2.5 MATLAB and Symbolic Computation in AI Systems

Engineering applications require precise numerical computation, which traditional LLMs struggle to perform accurately. MATLAB-based systems have been widely used for solving differential equations, control systems, and power system analysis.

Recent research integrates MATLAB execution within AI systems to bridge the gap between symbolic reasoning and numerical accuracy. These approaches ensure deterministic results, particularly in domains such as electrical engineering.

As shown in Table II.1, recent methods have advanced significantly, yet limitations remain in creating a single unified framework that resolves all aspects.

Table II.1: Summary of Recent Literature on AI Reasoning Systems

Ref.	Authors/Year	Methodology	Key Result	Limitation
[9]	Lewis et al., 2020	Retrieval-Augmented Generation (RAG)	Improved factual accuracy	Struggles with structured rules
[15]	Karpukhin et al., 2020	Dense Passage Retrieval (DPR)	Better retrieval performance	Lacks explicit relationships
[17]	Wang et al., 2017	KG-based Question Answering (KGQA)	High interpretability for queries	Weak handling of unstructured data
[13]	Pan et al., 2024	Unifying LLMs and Knowledge Graphs	Robust multi-hop reasoning	Integration complexity
[14]	Schick et al., 2023	Tool-Augmented LLMs (Toolformer)	Autonomous external API usage	Lacks coordinated task planning
[20]	Gou et al., 2023	Tool-Integrated Reasoning (ToRA)	High accuracy in math tasks	Computation treated as isolated
[19]	Park et al., 2023	Multi-Agent Generative Systems	Collaborative reasoning and planning	Complex conflict resolution
[8]	Wu et al., 2023	AutoGen Multi-Agent Framework	Modular conversational decomposition	Coordination and routing overhead

2.3 Summary

From the extensive literature review, it is evident that while significant progress has been made in improving LLM capabilities through retrieval, knowledge graphs, and tool integration, several critical gaps remain. Existing RAG-based systems struggle with structured reasoning and multi-hop queries in technical domains. Knowledge graph approaches provide strong relational reasoning but lack flexibility in handling unstructured data. Tool-augmented systems improve numerical accuracy but often operate without coordinated planning.

Furthermore, there is a clear lack of unified systems that integrate multi-agent orchestration, knowledge graphs, and external computational tools such as MATLAB within a single framework.

The proposed system addresses these gaps by developing a hybrid architecture that combines Knowledge Graph grounding, multi-agent coordination, and MATLAB-based execution to enable accurate, scalable, and domain-aware reasoning for electrical engineering and policy navigation applications.

III Proposed System Architecture

3.1 The Unified Engineering Team: A System Overview

To understand how this system works, imagine a high-tech engineering firm. In a standard firm, you wouldn't expect a single person to know every legal policy, perform every complex math calculation, and handle every customer email simultaneously. Instead, the firm is composed of specialized departments that work together under a manager. Our proposed system operates on this exact principle: it is a "multi-agent" framework where different digital specialists (agents) collaborate to solve problems that are too complex for a single AI to handle alone.

This modular design solves a critical problem in modern Artificial Intelligence: the tendency for Large Language Models¹ (LLMs) to "guess" when they encounter difficult math or specific policy navigation rules. By separating the "thinking" (linguistic reasoning) from the "doing" (executing math or searching databases), we create a system that is both flexible and factually grounded [8]. The internal workflow of this digital firm, from the moment a user asks a question to the second a final answer is delivered, is detailed in Figure III.1.

3.2 The Orchestrator: Task Triage and Management

At the entrance of our digital firm stands the Orchestrator, acting as the Project Manager. Its first job is "Task Triage" or Intent Classification². When you send a message, the Orchestrator quickly determines what kind of expert is needed. For instance, if the input is identified as simple conversation or *Small Talk*, it is handled directly by a natural language layer. If the query involves technical engineering problems, it is routed to the MATLAB specialist for rigorous calculation. Conversely, queries regarding policy navigation or factual data are directed to the Knowledge Agent for deep research. This systematic approach ensures that the system doesn't waste resources or use the wrong tool for the job.

Beyond just sorting, the Orchestrator maintains the "Project File" for the entire conversation. This is known as Context Synthesis³. It ensures that if you ask a follow-up question like "What about the second one?", the system knows exactly which item you are referring to by looking at the history. It rewrites your short questions into detailed instructions so that the specialist agents have all the background they need to work accurately without asking you

¹A type of AI trained on vast amounts of text to understand and generate human-like language.

²The process of identifying the purpose or goal behind a user's input.

³Combining information from previous parts of a conversation to understand current requests better.

to repeat yourself.

3.3 The Knowledge Agent: Accessing Policy Navigation Memory

The Knowledge Agent acts as the firm’s lead researcher. It has access to the “Policy Navigation Memory,” which we store in a Knowledge Graph⁴. Unlike a standard search engine that might just find keywords, this agent understands relationships. For example, it knows that a specific “Safety Valve” is part of a “Hydraulic System” and is governed by “Regulation 42.”

The primary effect of using this structured memory is the prevention of Hallucinations⁵. Instead of guessing a policy, the agent follows the “web” of facts to find a verified answer [13]. If the answer isn’t in the internal files, it performs an Iterative Search⁶ across the internet, cross-referencing external data with what it already knows to ensure the final report is comprehensive and 100% grounded in fact [9].

3.4 The MATLAB Agent: Expert Mathematical Solving

When a problem requires calculation, it is handed to the MATLAB Execution Agent—our digital Mathematician. Standard AIs often fail at math because they are predicting the next word rather than calculating a result. This agent avoids that trap by writing actual computer code and running it through professional-grade software [20].

To ensure the math is done correctly, the agent follows a strict blueprint known as a Pipeline. This involves a Topological Sort⁷ process, which ensures that each mathematical step is solved in the correct logical order—much like following a recipe where ingredients must be prepared before they can be cooked. Furthermore, every piece of code generated undergoes a Reviewer Loop, as visualized in the system flowchart. This process utilizes an “LLM Reviewer” to double-check the calculations, similar to having a second mathematician verify a colleague’s work on a chalkboard. If an error is detected, the script is corrected on the spot and re-executed until the result is verified as 100% accurate. This ensures that whether the system is calculating circuit resistance or complex fluid dynamics, the result is a product of verified logic, not a textual guess.

Table III.1: System Components: Analogies and Responsibilities

Component	The Analogy	What it does (Simple)	What it Prevents
Orchestrator	Project Manager	Sorts tasks and remembers the conversation history.	Confusion and Repetition
MATLAB Agent	Specialist Engineer	Writes and runs code for complex calculations.	Math Blunders
Knowledge Agent	Lead Researcher	Finds facts in digital maps and on the web.	Hallucinations

⁴A structured database that maps out entities and how they relate to one another, much like a giant digital web of facts.

⁵Instances where an AI generates information that sounds plausible but is factually incorrect.

⁶A process where the AI repeats its search several times, using what it learned in the first search to find more specific information.

⁷An algorithm used to order tasks so that each task is completed only after its dependencies are met (e.g., preparing ingredients before cooking).

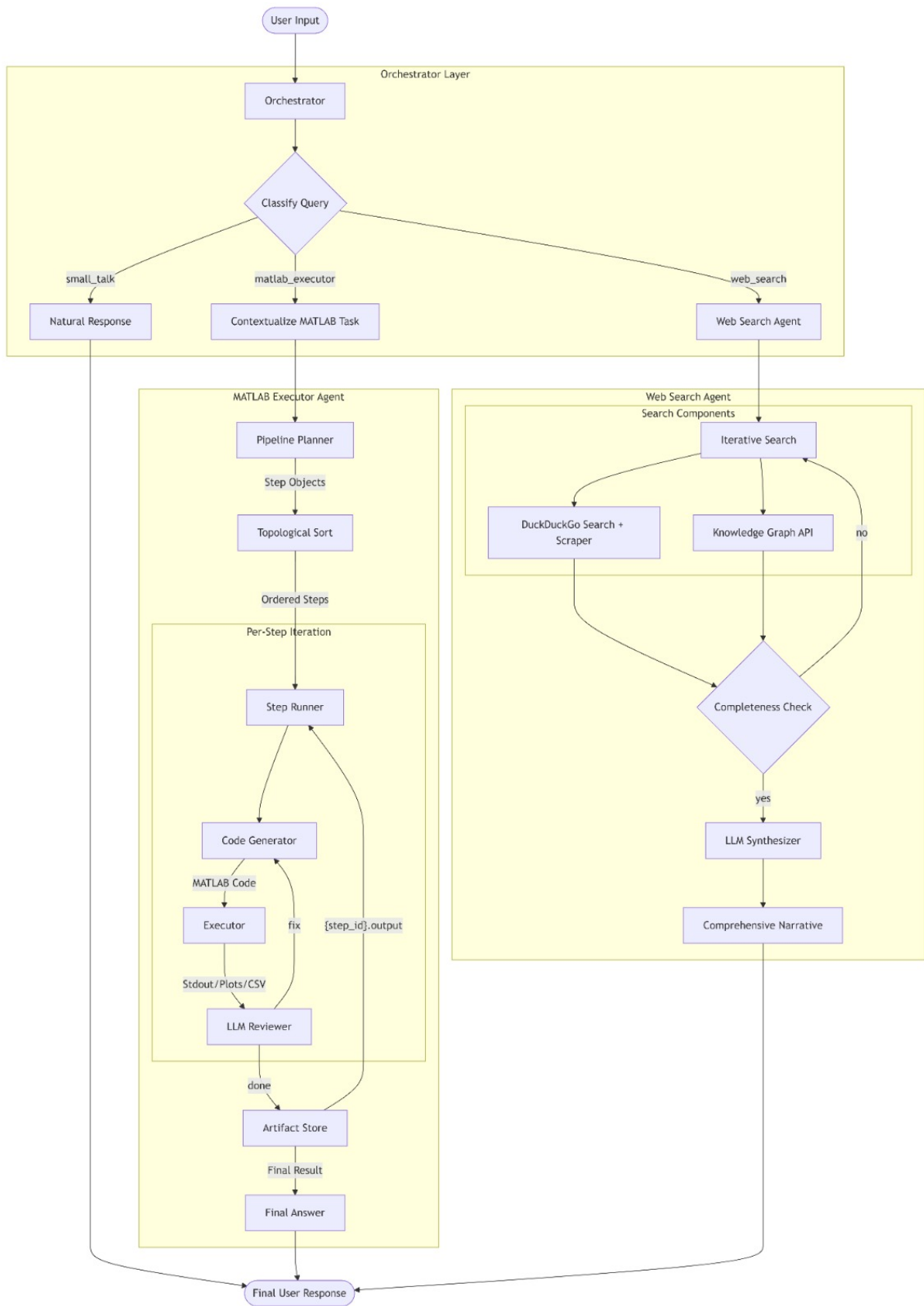


Figure III.1: Detailed System Architecture and Workflow

IV Methodology

4.1 Knowledge Graph Design and Construction

4.1.1 Extraction Strategy

The construction of the knowledge graph follows a multimodal extraction paradigm designed to handle the complexity of technical manuals and policy documents [12]. Rather than traditional text parsing, the system processes document pages as high-resolution images, allowing a vision-capable language model to interpret spatial relationships, flowcharts, and complex tables. This approach decomposes documents into autonomous knowledge units, which are semantically rich fragments of information that retain their meaning in isolation. By recursively identifying entities and their logical intersections across multiple pages, the system maintains a consistent and interconnected database of facts.

The philosophy behind this extraction strategy centers on the preservation of semantic context. Unlike traditional keyword-based methods, the multimodal approach captures the functional intent of the information, ensuring that technical specifications and procedural rules are stored with their necessary qualifiers. By passing previously identified entities back into the extraction loop, the system maintains a consistent taxonomy of concepts, preventing the redundant creation of nodes and ensuring that the final graph reflects a coherent and comprehensive understanding of the source material [13].

4.1.2 Graph Population

The population phase involves transforming the extracted knowledge units and their logical relationships into a structured graph format. Each identified entity is assigned a functional type based on its role within the document, such as a procedural step or a technical requirement. To facilitate efficient retrieval, these entities are converted into high-dimensional vector representations. The nodes and edges are then stored in a specialized database, where a coordinate-based indexing system is established to allow for rapid semantic searching and context-aware traversal of the information network.

To enhance the depth of the graph, the population process incorporates a vector-based embedding layer that allows for nuanced semantic retrieval. This enables the system to identify relevant information even when a direct textual match is absent, by calculating the mathematical proximity between the user’s intent and the stored knowledge

units. The resulting graph is a dynamic resource that supports both structured traversal of relationships and flexible semantic searches, providing a multi-layered foundation for the system’s reasoning capabilities.

4.1.3 Grounding and Entity Resolution

The primary effect of this Knowledge Graph construction is the elimination of hallucination through structural grounding [13]. During the transformation phase, entities extracted from disparate pages are resolved against a central taxonomy. This process ensures that a concept mentioned on page 10 of a technical manual is linked to its related safety constraint on page 150, creating a “single source of truth.” By shifting information retrieval from probabilistic searching to deterministic graph traversal, the system ensures that every policy navigation response is directly traceable to its source metadata, thereby providing a verifiable audit trail that is critical for policy navigation.

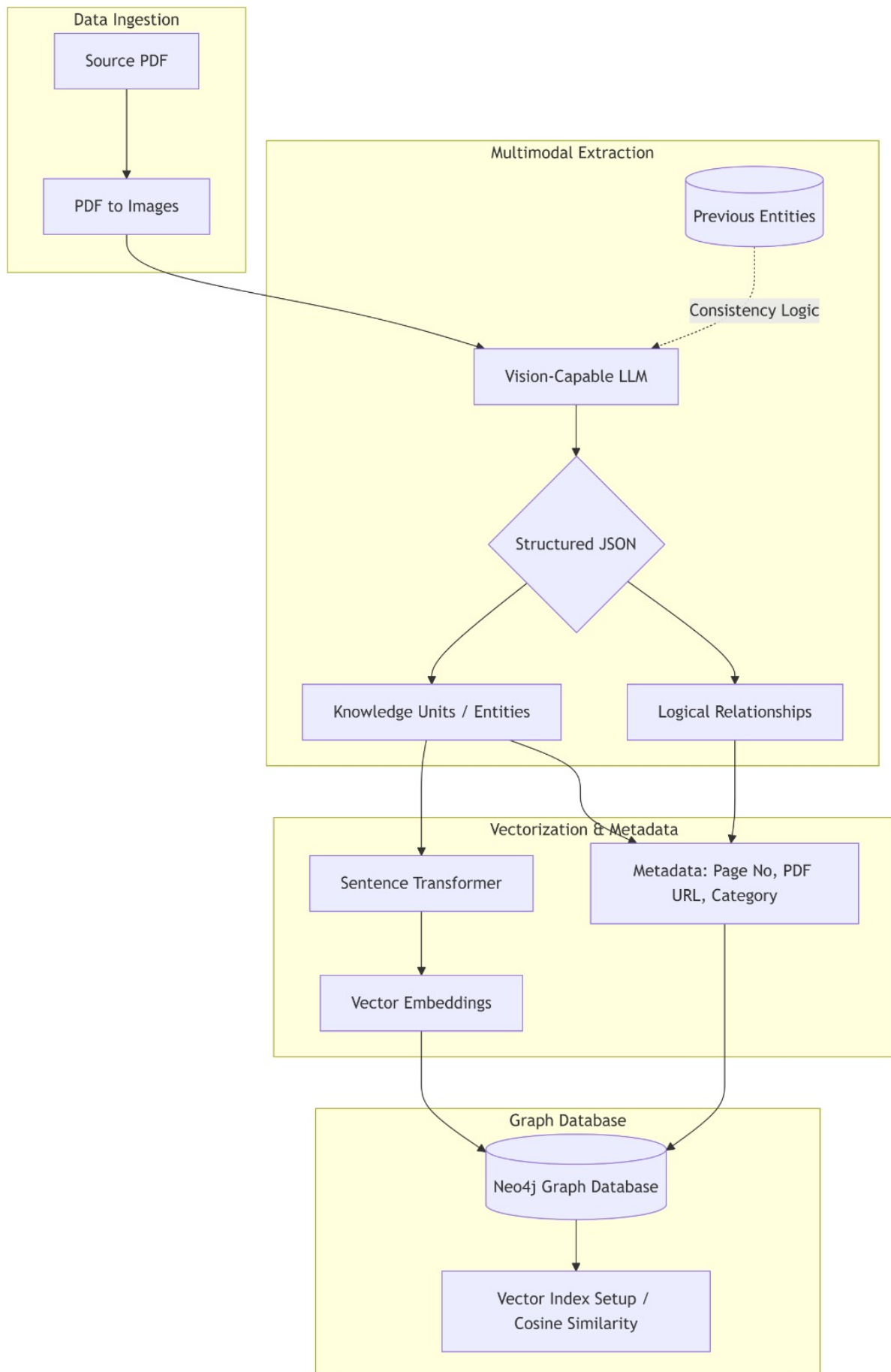


Figure IV.1: Knowledge Graph Flowchart

4.2 Query Processing Pipeline

The system processes natural language queries by first identifying the core concepts within the user’s request. These concepts are mapped to the graph database using a similarity-based search mechanism that compares the intent of the query with the stored knowledge units. Once relevant nodes are identified, the pipeline retrieves the surrounding network of relationships to provide a grounded context. This ensures that the final response is not merely a linguistic prediction but is derived from the structured hierarchy of rules and technical specifications stored within the graph.

The pipeline is designed to handle varying levels of query complexity, from single-fact lookups to multi-faceted analytical requests. By extracting entities from the user’s query and performing coordinated searches across both the graph and external resources, the system constructs a comprehensive context for the reasoning engine. This multi-stage retrieval process ensures that every response is backed by specific, cited evidence, reinforcing the system’s reliability and providing a transparent audit trail for the information being presented.

4.3 Multi-Agent Coordination

Coordination between specialized agents is managed through a routing logic that prioritizes task decomposition and conflict resolution [8]. When a query involves both engineering calculations and policy navigation constraints, the system ensures that the output of the mathematical agent is validated against the retrieval results of the policy agent. This collaborative intelligence allows the framework to handle multi-faceted problems, such as verifying if a calculated engineering specification complies with established safety protocols or policy navigation guidelines.

This coordination layer also serves as a final consistency check to prevent logical contradictions between different processing domains. By evaluating the outputs of each agent against a set of global constraints, the system can identify and resolve potential conflicts before the final response is generated. This cross-modal validation ensures that the final narrative is not only mathematically accurate but also administratively compliant, providing a unified and reliable solution to the user’s request.

4.4 Validation and Feedback Loops

To ensure the reliability of the system, multiple layers of validation are integrated into the processing workflow. For mathematical tasks, the system employs a sophisticated “Per-Step Iteration” methodology within the MATLAB Execution Agent. This process begins with the *Pipeline Planner*, which decomposes complex problems into atomic sub-tasks. For each sub-task, the *Code Generator* produces a MATLAB script which is then executed in a live environment. The results of this execution—including numerical outputs, plots, or error messages—are passed to an *LLM Reviewer*.

This reviewer acts as a secondary intelligence layer, evaluating the raw computational output against the original

mathematical intent. If a logical inconsistency or a syntax error is detected, the reviewer provides specific feedback for a “fix loop,” where the code is corrected and re-executed [21, 22]. This iterative self-correction mechanism continues until the result is verified as 100% accurate. The effect of this rigorous loop is the total elimination of computational drift; whereas standard language models often accumulate small errors across multi-step problems, our system remains deterministic, ensuring that engineering results are products of verified execution rather than textual approximation.

Similarly, the knowledge retrieval process uses an iterative refinement loop where the agent assesses the completeness of its findings [23]. If the retrieved data is insufficient to fully address the query, the system generates targeted follow-up searches until a satisfactory level of information density is achieved. These feedback loops ensure that the final output meets the high standards of accuracy required for technical and policy navigation applications.

V Implementation

5.1 Technical Stack

The implementation is built upon a high-performance Python-based framework that orchestrates communication between advanced reasoning models, structured data layers, and deterministic computation engines. The architecture prioritizes low-latency reasoning and high-fidelity grounding by leveraging the Ollama inference engine, which hosts the GPT-4 large language model as the system’s reasoning backbone. For policy-oriented data storage, a Neo4j graph database is utilized to maintain the hierarchical and relational integrity of policy navigation.

The system is deployed within a Docker-containerized environment to ensure scalability and consistent behavior across different computational infrastructures. This setup includes a localized embedding engine for vector-based semantic retrieval and a vision-capable processing layer for multimodal data ingestion. Table V.1 summarizes the core components and the specific technologies integrated into the platform.

Table V.1: Technical Stack Components

Component	Technology / Provider
Core Programming Language	Python 3.11+
Reasoning Engine (LLM)	Ollama Inference API
Graph Database	Neo4j (Cypher Query Language)
Numerical Computation	MATLAB (Subprocess Execution)
Search Interface	DuckDuckGo API (Iterative Search)
Containerization	Docker
Inter-Agent Communication	JSON-based messaging and CSV artifacts

5.2 Tool Integration and Workflow

Integration between the reasoning framework and external engines is achieved through a secure inter-process communication (IPC) bridge. This allows the system to interact with a dedicated MATLAB instance for symbolic and numerical computation without compromising the stability of the core application.

The integration logic follows a specific “Agentic Loop” for each domain, starting with the Orchestration Layer. At this stage, all user queries are first contextualized using conversation history and then classified to determine the optimal processing path. This ensures that technical tasks are routed to the specialized execution environments without loss of previous context.

For mathematical and engineering queries, the MATLAB Agent Pipeline triggers a four-stage process consisting of Planning, Topological Sorting, Code Generation, and Execution. The Planning phase decomposes complex problems into atomic steps, while Topological Sorting manages dependency structures. To ensure reliability, a “Self-Correction Loop” utilizes an LLM Reviewer to automatically identify and fix code errors if the MATLAB engine returns a runtime exception.

In the case of factual research, the Web Search Agent Loop enables iterative searching across DuckDuckGo and the local Knowledge Graph. The agent synthesizes its findings into a cited report only after a rigorous “completeness check” is passed, ensuring that the final output is comprehensive and grounded. This entire multi-step process is supported by an Artifact Management system, where intermediate results such as matrices, tables, or refined search queries are stored in temporary CSV files. This state management strategy allows the system to pass data between discrete pipeline steps without overwhelming the linguistic context window of the reasoning model.

By using standardized data formats like CSV and JSON, the system maintains a modular architecture that supports the rapid incorporation of new computational tools as requirements evolve.

VI Experimental Setup and Results

The evaluation of the proposed framework was divided into two distinct components: testing the MATLAB execution agent against traditional language models for mathematical and engineering tasks, and testing the system’s Knowledge Graph retrieval capabilities for policy navigation.

6.1 Testing with MATLAB as an Execution Tool

6.1.1 Evaluation Objectives

The primary objective of this evaluation phase was to assess the computational accuracy, reliability, and computational efficiency of the MATLAB execution agent when compared to a traditional large language model (GPT-5.4). Specifically, the testing aimed to determine whether execution-based reasoning—generating and running MATLAB scripts—can overcome the inherent limitations of next-token prediction in complex mathematical and Electrical Engineering (EEE) tasks.

6.1.2 Dataset and Query Design

To rigorously evaluate the mathematical capabilities of the agent, a benchmark dataset consisting of 100 calculation-based engineering queries was constructed. These queries were specifically curated to evaluate deterministic numerical reasoning and were categorized into three levels of difficulty: Simple, Medium, and Complex. This categorization was based on the depth of reasoning required and the number of intermediate computational steps.

Simple queries primarily involved direct computations with minimal mathematical overhead, such as basic numerical trigonometry. Medium-level queries introduced multi-step dependencies requiring the tracking of intermediate states. Complex queries represented advanced problems necessitating chained computations, such as matrix adjoints and iterative Newton-Raphson methods, which significantly increase the cognitive load on standard LLMs. This comprehensive dataset ensures that both shallow and deep execution capabilities are thoroughly examined.

Topic	Simple	Medium	Complex	Total
Numerical Trigonometry	7	9	9	25
Advanced Arithmetic	14	12	9	35
Linear Algebra	6	8	6	20
Newton-Raphson	6	8	6	20
Total	33	37	30	100

Table VI.1: Distribution of mathematical questions by topic and difficulty

6.1.3 Performance Benchmarking

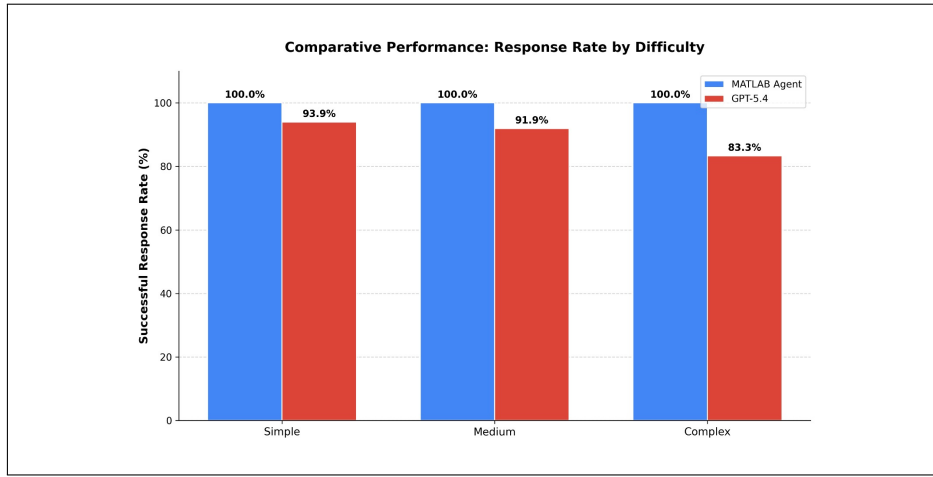


Figure VI.1: Comparative performance across difficulty levels

The benchmarking results reveal a clear and consistent performance advantage of the proposed MATLAB-based execution agent over traditional large language model-based approaches. Across all levels of difficulty, the agent achieves a perfect success rate of 100%, demonstrating robustness and reliability in handling both simple and complex tasks. In contrast, GPT-5.4 exhibits a progressive decline in performance as task complexity increases, with accuracy decreasing from 93.9% for simple queries to 83.3% for complex queries.

This degradation can be attributed to the inherent limitations of probabilistic sequence generation in language models, where errors in intermediate reasoning steps accumulate and propagate through the reasoning chain. The execution-based paradigm adopted in this work effectively mitigates this issue by grounding computations in deterministic logic, enforcing the principle that execution accuracy reliably exceeds prediction accuracy:

$$\text{Accuracy}_{\text{execution}} > \text{Accuracy}_{\text{prediction}} \quad (\text{VI.1})$$

6.1.4 Computational Efficiency

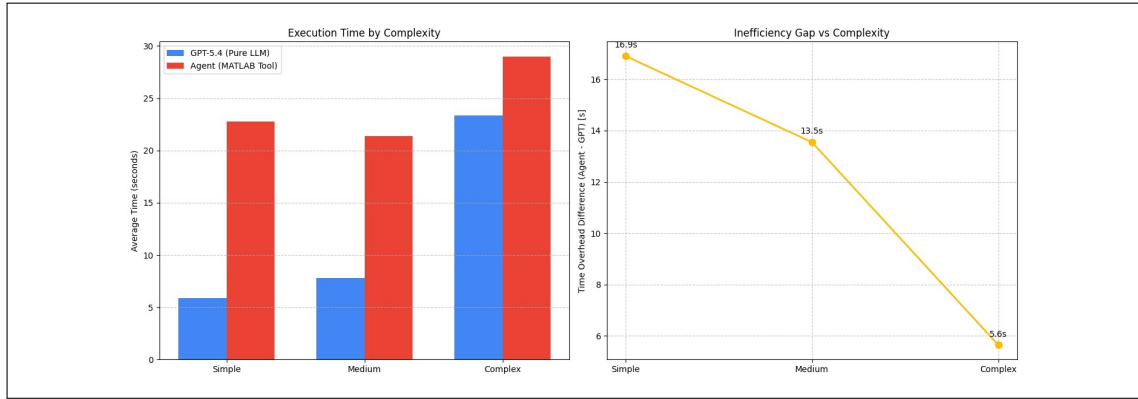


Figure VI.2: Execution time comparison and inefficiency gap vs complexity

The computational efficiency analysis highlights a critical trade-off between response latency and computational accuracy. For simple queries, GPT-5.4 demonstrates lower latency due to the absence of external tool invocation. However, the MATLAB-based agent incurs additional overhead due to script generation, execution, and validation processes, resulting in higher initial response times.

Despite this initial disadvantage, the inefficiency gap between the two approaches decreases substantially as task complexity increases, reducing from approximately 16.9 seconds for simple tasks to 5.6 seconds for complex tasks. This indicates that while the execution overhead remains relatively constant, the reasoning burden on standard LLM-based systems increases disproportionately. This trend demonstrates that execution-based systems scale more effectively with mathematical problem difficulty.

6.2 Testing Knowledge Graph Retrieval with NIT Andhra Pradesh Policies

6.2.1 Evaluation Objectives

The second phase of evaluation focused on assessing the Knowledge Graph (KG) retrieval capabilities of the proposed system when navigating complex, structured, and cross-referenced policy documents. This phase aimed to measure how effectively the system can accurately retrieve domain-specific facts and trace logical dependencies, comparing this structurally-grounded retrieval against the context fragmentation and hallucination often seen in baseline LLMs.

6.2.2 Dataset and Query Design

A specialized benchmark was manually curated based on the NIT Andhra Pradesh rule book, encompassing various policy navigation rules, including alcohol policies, promotion criteria, and grading systems. To systematically

evaluate varying retrieval complexity, the queries were structured into three specific typologies. Single Needle queries required the extraction of a direct, singular fact from the knowledge base, such as identifying the specific act linked to a harassment policy. Multi-Needle queries demanded the extraction of multiple, somewhat disjointed facts within a similar context, such as listing down all the cases where a student can receive a specific administrative grade. Multi-Hop queries represented the most challenging category, requiring sequential traversal across interconnected entities where the output of one reasoning step serves as the input for the next, such as identifying a chained sequence of disciplinary fines under an alcohol policy.

This layered categorization ensures that the framework’s ability to handle deeper graph traversals and structured relationship tracking is comprehensively tested. Representative examples of these queries and their corresponding retrieval types are categorized in Table VI.2.

Sample Query	Retrieval Type
Who is the imposing authority of Littering on the campus and what other things do he oversee?	Multi-Hop
Under alcohol policy which category has a fine of 5000rs on 2nd offence and what is the fine for the same category for 2nd offence?	Multi-Hop
What is the act linked to harassment policy?	Single Needle
List down all the cases where a student can get an X grade.	Multi-Needle
Explain promotion criteria for all the years and the drug policy.	Multi-Hop & Multi-Needle

Table VI.2: Representative Knowledge Graph queries mapping policy navigation

6.2.3 Retrieval Performance and Analysis

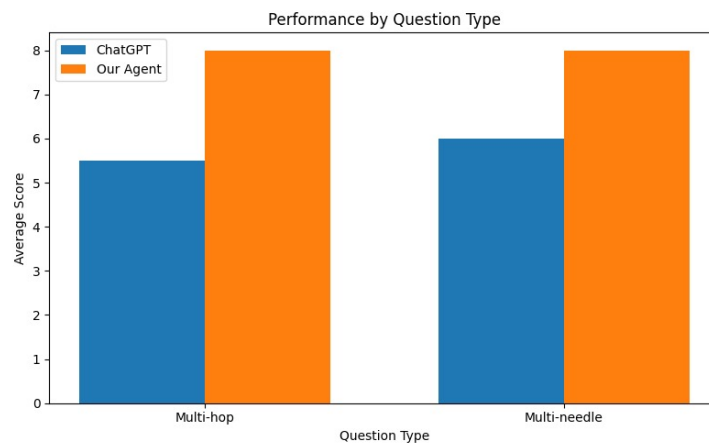


Figure VI.3: Knowledge Graph retrieval benchmark across specific query types

Complex reasoning tasks, particularly Multi-Hop and Multi-Needle queries, provide a critical evaluation of the system’s ability to navigate distributed policy knowledge. The benchmark results demonstrate that the proposed system consistently outperforms baseline generative LLMs when handling the dense, policy navigation guidelines of NIT Andhra Pradesh.

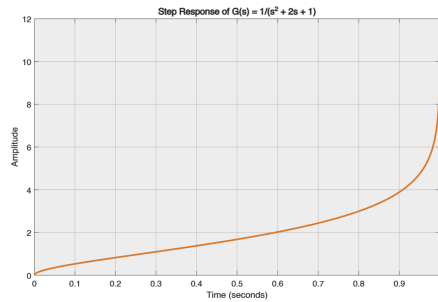
This improvement is primarily attributed to the explicit structural relationships encoded within the Knowledge Graph. By representing interconnected rules and conditions as deterministic paths, the system can reliably trace complex policy interactions without missing critical exceptions. Unlike traditional models which rely on implicit associations and are prone to semantic confusion when reading long manuals, the Knowledge Graph guarantees completeness of the retrieved evidence.

6.2.4 Error Analysis and Key Findings

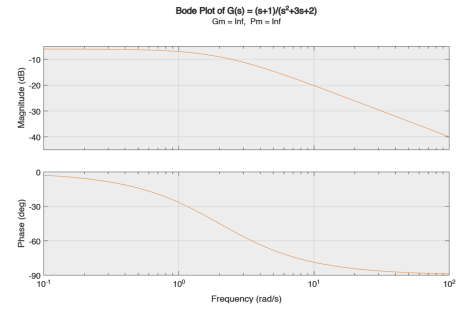
A detailed review of system failures during policy retrieval revealed that errors were primarily tied to the extraction phase and graph completeness rather than the reasoning engine itself. When the underlying Knowledge Graph coverage was incomplete, or there was ambiguity within the source administrative documents, the system occasionally struggled to establish clear relational mappings.

However, when correctly supported by a populated graph, the system eliminated standard model hallucinations. The final experimental results confirmed that separating logic execution into specialized tools significantly boosts stability: grounding the system via the Knowledge Graph ensures precise factual consistency for policy navigation, while execution-based computation maintains strict numerical accuracy for engineering problems.

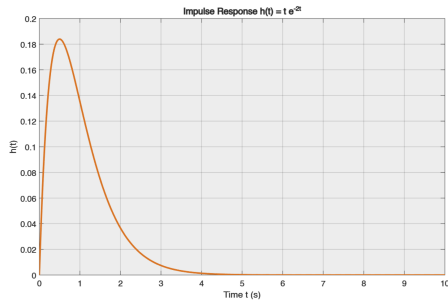
(a) Plot the step response of a system with the transfer function $G(s) = 1 / (s^2 + 2s + 1)$ using MATLAB.



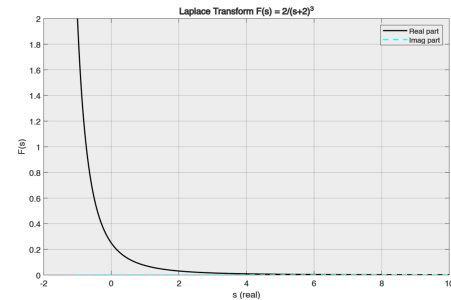
(b) Use MATLAB to create a Bode plot for the system $G(s) = (s + 1) / (s^2 + 3s + 2)$ and analyze its stability.



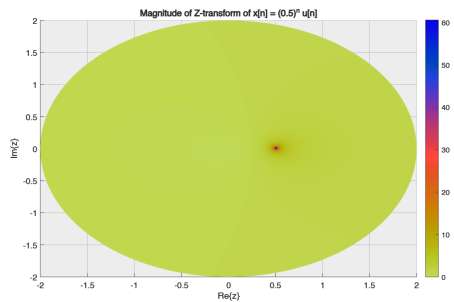
(c) Write a MATLAB script to plot the impulse response of the system $y''(t) + 4y'(t) + 4y(t) = u(t)$, where $u(t)$ is the unit impulse function.



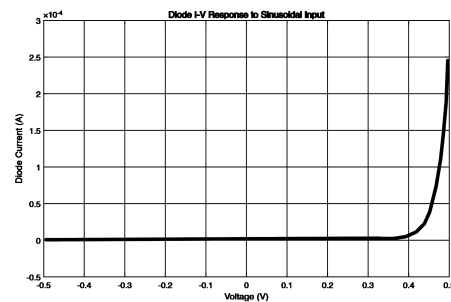
(d) Plot the Laplace transform of the function $f(t) = t^2 * e^{-2t}$ using MATLAB and identify its poles and zeros.



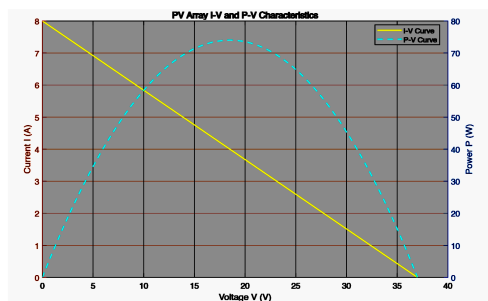
(e) Create a plot in MATLAB that shows the Z-transform of the sequence $x[n] = (0.5)^n * u[n]$.



(f) Use MATLAB to plot the response of a diode to a sinusoidal input, assuming the Shockley diode equation.



(g) Plot the PV array characteristics, including the current-voltage (I-V) and power-voltage (P-V) curves, using MATLAB.



(h) Write a MATLAB code snippet to generate a Nichols plot for the system $G(s) = (s + 2) / (s^2 + 2s + 1)$ and interpret the results.

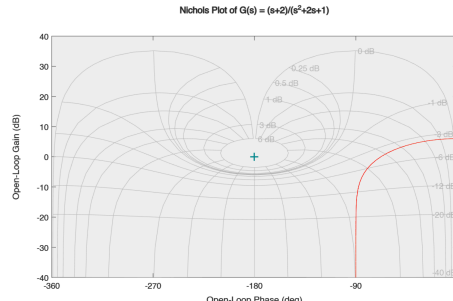


Figure VI.4: MATLAB plots of various questions

VII Discussions

7.1 Conclusions

This work presents a comprehensive approach to enhancing reasoning accuracy and computational reliability by integrating Knowledge Graph grounding with execution-based agents. The proposed architecture demonstrates a significant advancement over conventional language model-based systems by effectively integrating deterministic computation with structured knowledge retrieval.

The system addresses the fundamental limitations of traditional large language models, particularly their susceptibility to cumulative reasoning errors and their inability to guarantee numerical precision. Unlike purely generative models that rely on probabilistic token prediction, the MATLAB-based execution agent ensures that all numerical computations are carried out with complete precision. This capability is especially critical in domains such as electrical engineering, where minor operational deviations can lead to completely invalid outcomes. Experimental results clearly reflect this, demonstrating that the execution-based approach maintains perfect accuracy across all tested mathematical complexities, whereas traditional LLMs degrade as problem depth increases.

Another prominent strength lies in the system’s ability to perform structured multi-hop reasoning. By utilizing a Knowledge Graph, the framework can systematically traverse interconnected entities and relationships scattered across complex policy navigation documents. This structured network dramatically reduces hallucination and limits the likelihood of missing intermediate steps—a common failure point in purely text-fragment-based retrieval systems.

However, despite strong architectural performance, the proposed system is subject to specific limitations arising from component integration. Using external computational engines like MATLAB inherently increases response latency, introducing overhead related to code generation, execution loops, and syntax validation. While the latency tradeoff is highly justified for complex problems, it limits efficiency for simple queries. Additionally, the system remains heavily dependent on the absolute completeness and accuracy of the Knowledge Graph; missing structural relationships or contradictory statements within source rulebooks can impair retrieval and resolution. The multi-agent routing layer also serves as a potential point of failure if user intent is incorrectly classified.

Ultimately, by fusing natural language understanding with specialized, rule-based execution environments, the framework provides a robust foundation for future scalable AI implementations. Its modular nature allows for potential transitions into broader engineering fields or enterprise resource planning (ERP) platforms, positioning

it as a highly adaptable solution for tasks demanding absolute precision and logical clarity.

7.2 Future Scope

While the current framework demonstrates reliable performance, several key avenues remain for substantial improvement and real-world scalability.

One of the most critical directions for future work is the development of dynamic Knowledge Graph updating mechanisms. Integrating automated workflows that can incrementally ingest and update nodes as policy navigation requirements evolve would allow the system to adapt in real time, a crucial feature for enterprise compliance environments.

Additionally, the system’s orchestration and planning capabilities require further refinement. Incorporating advanced machine learning models for query routing, alongside predictive caching of frequently executed mathematical functions, would drastically optimize latency and reduce reliance on repeated API calls to external execution environments. Moving towards parallel execution for independent reasoning forks could also limit response times significantly.

Finally, expanding the system’s external integration will unlock dynamic monitoring capabilities. Connecting the execution agent to real-time data streams—such as live sensor feeds, SCADA systems, or enterprise databases—would transition the framework from a conversational question-answering assistant into an active, operational decision-support tool. Implementing these optimizations will cement the system’s viability as a general-purpose intelligent framework capable of unifying symbolic computation with state-of-the-art semantic reasoning.

REFERENCES

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.
- [2] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.
- [3] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.
- [4] V. Rawte, A. Sheth, and A. Das, “A survey of hallucination in large foundation models,” *arXiv preprint arXiv:2309.05922*, 2023.
- [5] F. Amjad, T. Korötko, and A. Rosin, “Review of LLMs applications in electrical power & energy systems,” *IEEE Access*, vol. 13, pp. 7669–7687, 2025.
- [6] S. Majumder, L. Dong, F. Doudi, Y. Cai, C. Tian, D. Kalathil, K. Ding, A. A. Thatte, N. Li, and L. Xie, “Exploring the capabilities and limitations of large language models in the electric energy sector,” *Joule*, vol. 8, no. 6, pp. 1544–1549, 2024.
- [7] G. Mialon, R. Dessì, M. Lomeli, C. Nalmpantis, R. Pasunuru, R. Raileanu, B. Rozière, N. Goyal, A. Joulin, and T. Scialom, “Augmented language models: a survey,” *arXiv preprint arXiv:2302.07842*, 2023.
- [8] Q. Wu, G. Bansal, J. Zhang, Y. Wu, S. Zhang, E. Zhu, B. Jiang, X. Li, S. Cho, C. Wang *et al.*, “Autogen: Enabling next-gen llm applications via multi-agent conversation,” *arXiv preprint arXiv:2308.08155*, 2023.
- [9] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [10] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 24 824–24 837.

- [11] S. Imani, L. Du, and H. Shrivastava, “Mathprompter: Mathematical reasoning using large language models,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 5: Industry Track)*, 2023, pp. 37–42.
- [12] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R. O. Ness, and J. Larson, “From local to global: A graphrag approach to query-focused summarization,” *arXiv preprint arXiv:2404.16130*, 2024.
- [13] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu, “Unifying large language models and knowledge graphs: A roadmap,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 7, pp. 3580–3599, 2024.
- [14] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” *arXiv preprint arXiv:2302.04761*, 2023.
- [15] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, “Dense passage retrieval for open-domain question answering,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781.
- [16] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, “Retrieval-augmented generation for large language models: A survey,” *arXiv preprint arXiv:2312.10997*, 2023.
- [17] X. Wang *et al.*, “Knowledge-based question answering: A review,” *arXiv preprint*, 2017.
- [18] Z. Sun, Z.-H. Deng, J.-Y. Nie, and J. Tang, “Rotate: Knowledge graph embedding by relational rotation in complex space,” in *International Conference on Learning Representations*, 2019.
- [19] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative agents: Interactive simulacra of human behavior,” in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023, pp. 1–22.
- [20] Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, M. Huang, N. Duan, and W. Chen, “Tora: A tool-integrated reasoning agent for mathematical problem solving,” 2023.
- [21] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 6710–6726.
- [22] X. Chen, M. Lin, N. Schärli, and D. Zhou, “Teaching large language models to self-debug,” *arXiv preprint arXiv:2304.05128*, 2023.
- [23] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhume, Y. Yang, S. Welleck, B. P. Majumder, S. Gupta, A. Yazdanbakhsh, and P. Clark, “Self-refine: Iterative refinement with self-feedback,” 2023.

BIO-DATA

Student 1

Name : Mehul Jain
Roll No : 522206
Date of Birth : 17/06/2002
Address : 168-169 Street no. 7, Sadar Bazar Jabalpur
Email : mehulj475@gmail.com
Education : B.Tech in Electrical and Electronics Engineering, NIT
Andhra Pradesh (2022-2026)



Student 2

Name : Krishna Tayal
Roll No : 522152
Date of Birth : 29/09/2003
Address : 1366, Sector-7 Extension, Gurugram
Email : krishnatayal05@gmail.com
Education : B.Tech in Electrical and Electronics Engineering, NIT
Andhra Pradesh (2022-2026)



Student 3

Name : Ramasish Parida
Roll No : 522232
Date of Birth : 26/03/2003
Address : Sector-1 , Rourkela , Odisha
Email : ramasishparida@gmail.com
Education : B.Tech in Electrical and Electronics Engineering, NIT
Andhra Pradesh (2022-2026)

